

Formal Languages and Compilers

Prof. Breveglieri, Morzenti, Agosta

Written exam: laboratory question

25/01/2024

Time: 60 minutes. Textbooks and notes can be used. Pencil writing is allowed.

Important: Write your name on any additional sheet.

SURNAME (Cognome):

NAME (Nome):

Matricola:or Person Code:

Instructor: ☐ Prof. Breviglieri ☐ Prof. Morzenti ☐ Prof. Agosta

The laboratory question must be answered taking into account the implementation of the Acse compiler given with the exam text.

Modify the specification of the lexical analyser (flex input) and the syntactic analyser (bison input) and any other source file required to extend the Lance language with the ability to evaluate **iterated expressions**.

This operation is available through a new statement called **repeat_exp**, having the following syntax:

repeat_exp ($\langle var. \rangle = \langle exp. 1 \rangle, \langle exp. 2 \rangle, \langle exp. 3 \rangle$);

The `repeat_exp` statement implements a simple loop which, at every iteration, re-computes $\langle exp. 3 \rangle$ and assigns it to $\langle var. \rangle$. $\langle var. \rangle$ is initialized to the value of $\langle exp. 1 \rangle$ before the loop. The number of iterations is controlled by the value of $\langle exp. 2 \rangle$.

If $\langle exp. 2 \rangle$ is zero or negative, the loop should never iterate, leaving $\langle var. \rangle$ to its initial value, i.e. $\langle exp. 1 \rangle$. Even though this is allowed by the syntax, assume that $\langle exp. 2 \rangle$ does not use $\langle var. \rangle$ in its definition. As a result you can assume that $\langle exp. 2 \rangle$ doesn't change during the execution of the loop.

The following code snippet exemplifies the operation of the `repeat_exp` statement. The first `repeat_exp` statement initializes a to zero, then performs the assignment $a \leftarrow a + 1$ ten times — as specified by the $\langle exp. 2 \rangle$ argument — leaving $a = 10$ at the end. The second `repeat_exp` initializes b to $a \times 3 = 30$, but then $\langle exp. 2 \rangle$ evaluates to -5 , which is a negative value. As a result, b is left equal to 30 and is not changed again. The third occurrence of `repeat_exp` implements a variant of the well-known recurrent formula $x_{i+1} = \frac{1}{2} \left(x_i + \frac{N}{x_i} \right)$ to compute $\sqrt{4096576}$.

```
int a, b;

repeat_exp(a=0, 10, a+1);
// a == 10

repeat_exp(b=a*3, a-15, b-1);
// b == 30

repeat_exp(a=1000, 3, (a+4096576/a)/2);
// a == 2024
```

1. Define the tokens (and the related declarations in **Acse.lex** and **Acse.y**). (1 points)
2. Define the syntactic rules or the modifications required to the existing ones. (2 points)
3. Define the semantic actions needed to implement the required functionality. (22 points)

Solution

The solution is shown below in *diff* format. All lines that begin with '+' were added, while the lines that begin with '-' were removed. **It is not required and it is not encouraged to provide a solution in *diff* format to get the maximum grade.**

```
diff --git a/acse/Acse.lex b/acse/Acse.lex
index 35f9b73..bff94d9 100644
--- a/acse/Acse.lex
+++ b/acse/Acse.lex
@@ -93,6 +93,7 @@ ID      [a-zA-Z_][a-zA-Z0-9_]*
"return"      { return RETURN; }
"read"        { return READ; }
"write"       { return WRITE; }
+"repeat_exp" { return REPEAT_EXP; }

{ID}          { yyval.svalue=strdup(yytext); return IDENTIFIER; }
{DIGIT}+      { yyval.intval = atoi( yytext ); }
diff --git a/acse/Acse.y b/acse/Acse.y
index 5e31c71..1197d28 100644
--- a/acse/Acse.y
+++ b/acse/Acse.y
@@ -109,6 +109,7 @@ extern void yyerror(const char*);
    t_list *list;
    t_axe_label *label;
    t_while_statement while_stmt;
+   t_repeat_exp_statement rep_exp;
}
/*=====
                                TOKENS
@@ -133,6 +134,7 @@ extern void yyerror(const char*);
%token <intval> TYPE
%token <svalue> IDENTIFIER
%token <intval> NUMBER
+%token <rep_exp> REPEAT_EXP

%type <expr> exp
%type <decl> declaration
@@ -248,6 +250,66 @@ statement : assign_statement SEMI { /* does nothing */ }
           | control_statement     { /* does nothing */ }
           | read_write_statement SEMI { /* does nothing */ }
           | SEMI                  { gen_nop_instruction(program); }
+          | repeat_exp_statement SEMI
+;
+;
+repeat_exp_statement: REPEAT_EXP LPAR IDENTIFIER ASSIGN exp COMMA exp COMMA
+{
+    // Generate the assignment of <exp. 1> to the variable.
+    int r_var = get_symbol_location(program, $3, 0);
+    if ($5.expression_type == REGISTER) {
+        gen_addi_instruction(program, r_var, $5.value, 0);
+    } else {
+        gen_addi_instruction(program, r_var, REG_0, $5.value);
+    }
+;
+    // Reserve a new register which will be used as a loop counter, and
+    // generate the code which initializes the loop counter with
+    // the value of <exp. 2>.
+    // The loop counter will be decremented at every iteration.
+    $1.r_i = getNewRegister(program);
+    if ($7.expression_type == REGISTER) {
```

```

+         gen_addi_instruction(program, $1.r_i, $7.value, 0);
+     } else {
+         gen_addi_instruction(program, $1.r_i, REG_0, $7.value);
+     }
+
+     // Generate the label for the back-edge of the loop
+     $1.l_loop = assignNewLabel(program);
+     // Generate the code for testing if the loop counter is <= 0 and exit the
+     // loop in that case
+     gen_subi_instruction(program, REG_0, $1.r_i, 0);
+     $1.l_exit = newLabel(program);
+     gen_ble_instruction(program, $1.l_exit, 0);
+     // At this point the rule for 'exp' will generate some code which computes
+     // the value of <exp. 3>. If <exp. 3> is constant, no code is generated,
+     // and the same value will be assigned multiple times to the variable,
+     // which of course is inefficient.
+     // However, the text of the exercise didn't ask us to police the code
+     // we are compiling, so we shall *not* stop compilation in this situation.
+     // At most we could emit a warning, but *not* an error!
+ }
+ exp RPAR
+ {
+     // Generate code which will assign the value of <exp. 3> of the variable
+     // at each loop iteration.
+     int r_var = get_symbol_location(program, $3, 0);
+     if ($10.expression_type == REGISTER) {
+         gen_addi_instruction(program, r_var, $10.value, 0);
+     } else {
+         gen_addi_instruction(program, r_var, REG_0, $10.value);
+     }
+
+     // Generate code to decrement the loop counter and jump back at the head
+     // of the loop
+     gen_subi_instruction(program, $1.r_i, $1.r_i, 1);
+     gen_bt_instruction(program, $1.l_loop, 0);
+     // Generate the label which points just after the loop for exiting it.
+     assignLabel(program, $1.l_exit);
+
+     // Free the identifier
+     free($3);
+ }
+
+ ;

control_statement : if_statement { /* does nothing */ }
diff --git a/acse/axe_struct.h b/acse/axe_struct.h
index 6e7b689..f1b3c01 100644
--- a/acse/axe_struct.h
+++ b/acse/axe_struct.h
@@ -107,6 +107,17 @@ typedef struct t_while_statement
     * that follows the while construct */
 } t_while_statement;

+// Note that instead of defining a new semantic type in the union declaration,
+// we could have used global variables as the repeat_exp statement structurally
+// cannot be nested (being a statement, it cannot appear inside one of the
+// expressions!)
+typedef struct t_repeat_exp_statement
+{
+    int r_i;
+    t_axe_label *l_loop;
+    t_axe_label *l_exit;
+} t_repeat_exp_statement;
+
+/* create a label */
+extern t_axe_label *alloc_label(int value);

diff --git a/tests/exp_loop/exp_loop.src b/tests/exp_loop/exp_loop.src
new file mode 100644
index 0000000..92689a3

```

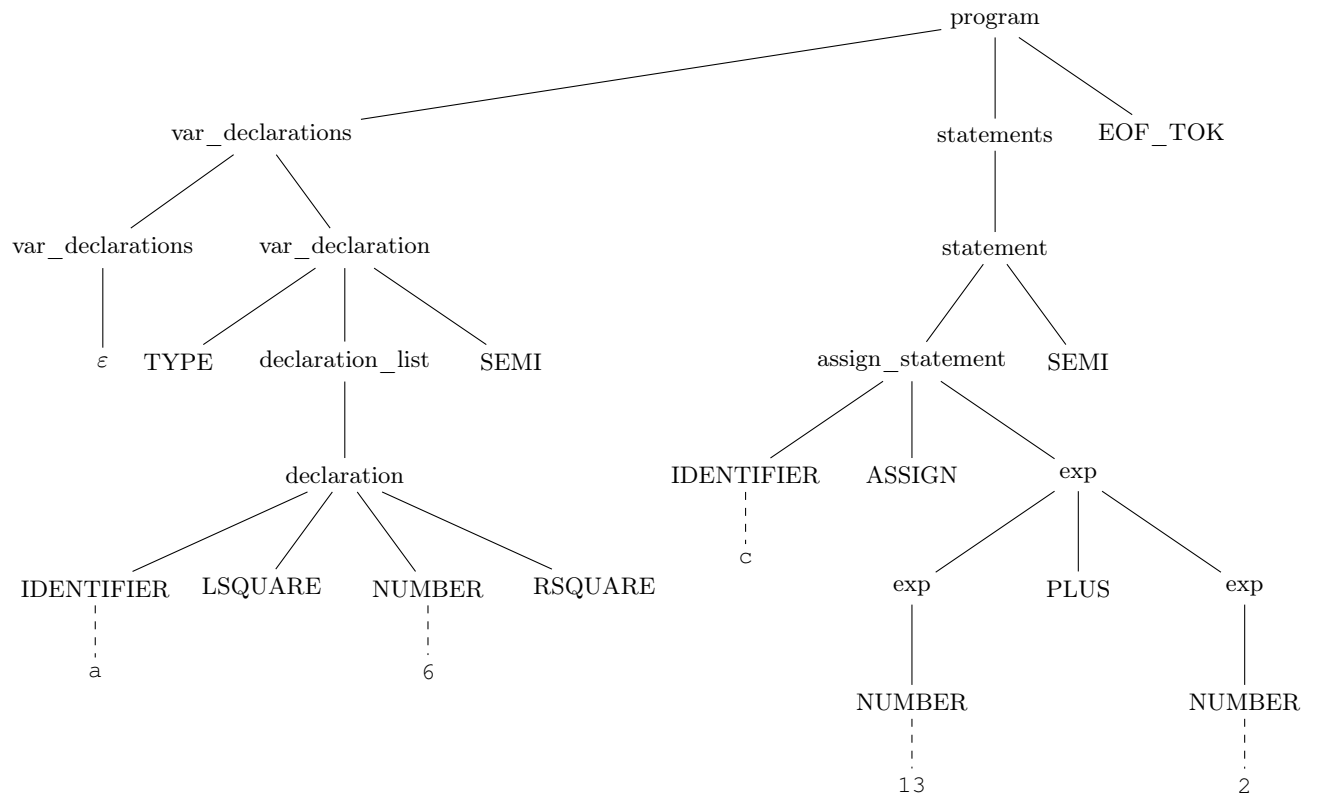
```
--- /dev/null
+++ b/tests/exp_loop/exp_loop.src
@@ -0,0 +1,16 @@
+int a, b;
+
++repeat_exp(a=0, 10, a+1);
+write(a); // 10
+
++repeat_exp(b=a*3, a+5, b-1);
+write(b); // 15
+
++repeat_exp(a=b/3, 0, 123456);
+write(a); // 5
+
++repeat_exp(a=10000, 3, (a+200000000/a)/2);
+write(a); // 14142
+
++repeat_exp(a=1000, 3, (a+4096576/a)/2);
+write(a); // 2024
```

4. Given the following Lance code snippet:

```
int a[6]; c=13+2;
```

write down the syntactic tree generated during the parsing with the Bison grammar described in Acse.y *starting from the axiom*. (5 points)

Solution



5. (**Bonus**) Briefly explain in plain English words how to modify the solution given to the questions about the `repeat_exp` statement in order to stop the loop earlier if the value of the variable stays the same for at least two consecutive iterations.